Chakra OS - User Guide

How to build Chakra, run it on a PC emulator, a real PC from USB and an Android phone, how to use it, what it is for, the shell commands, and how it compares with nanobyte.

What Chakra can be used for

Chakra 0.1.0 is a small, real operating system. It is not a replacement for Windows, Linux or Android, and it is honest to say so up front. Its value is that it is **yours, small enough to read in one sitting, and it really boots**. This page covers what it is good for, what you can do with it yourself, and what it cannot do.

What it is good for

1. Learning how an operating system works

Every part of a real OS is here in miniature, and each part fits in one short file:

You want to understand	Read this file
What happens when a computer powers on	`boot/stage1.asm`, `boot/stage2.asm`
Real mode, protected mode, the GDT and A20	`boot/stage2.asm`
Reading files from a FAT12 disk	`boot/stage2.asm`, `kernel/fat12.c`
How a kernel starts and runs C code	`kernel/entry.asm`, `kernel/kmain.c`
How the screen really works (video memory)	`kernel/vga.c`
How keys reach a program (keyboard controller)	`kernel/kbd.c`
Interrupts, the timer and CPU exceptions	`kernel/idt.c`
Memory: pages, paging and a heap	`kernel/mm.c`
Drawing a desktop with windows and a mouse	`kernel/gfx.c`, `kernel/gui.c`, `kernel/mouse.c`
How a network stack works	`kernel/net.c`, `kernel/rtl8139.c`, `kernel/http.c`
How USB devices are discovered	`kernel/uhci.c`
Tasks and context switching	`kernel/sched.c`, `kernel/entry.asm`
Talking to a disk and reading FAT12	`kernel/ata.c`, `kernel/fat12.c`

You want to understand	Read this file
How `printf` is built with no library	`kernel/lib.c`
How a command shell parses a line	`kernel/shell.c`

You can change any line, rebuild in seconds, and see the result. That makes it a safe place to make mistakes.

2. Teaching and demonstrations

- Show a class or a group what "booting" means by starting an emulator and watching Chakra come up.
- Use the red panic screen on purpose (a divide-by-zero in a test command) to explain CPU exceptions.
- Use it as a lab: "add a command that does X" is a good 20-minute exercise.

3. A foundation for your own bigger OS

Chakra already has a two-stage loader, interrupts, a memory manager, multitasking, a disk file system, a shell and a text UI. The roadmap lists what to add next: an ELF program loader, user mode, subdirectories, networking, graphics. Each step builds on what exists.

4. Low-level experiments

Because nothing sits between your code and the hardware, Chakra is a good test bench for:

- reading hardware ports (the real-time clock, PIT timer, keyboard controller, serial port)
- writing to video memory directly
- measuring how fast loops and system pieces run (using the timer ticks)
- trying assembly and C inline assembly

5. A showcase project

A from-scratch OS with its own bootloader, memory manager, scheduler and file system is a strong portfolio piece. The docs site in `docs/site/` is ready to show beside it.

What it can do today

Area	Today
Boot	Two-stage boot from a FAT12 disk: hard disk, floppy image or USB stick (BIOS mode)
Display	A **graphical desktop** with windows, a taskbar and a mouse; or 80x25 text mode with a status bar
Apps	Terminal, Files (file manager), Disks (disk manager), Editor, Browser, Monitor
Input	Keyboard on interrupts with arrow keys, PS/2 mouse, or characters over a serial port

Area	Today
Command line	Editing, history, tab completion, redirection (`>` and `>>`), about 60 commands
Memory	Page allocator, paging with a NULL-pointer guard, a kernel heap
Multitasking	Preemptive tasks, background jobs, a desktop task
Files and disks	FAT12 on an ATA disk, read and write, survives reboot; check, repair, label, format
Internet	DHCP, DNS, ping, TCP, HTTP, a text web viewer (Realtek RTL8139 card; plain http only)
USB and Bluetooth	USB controller (UHCI) driver and device listing; a Bluetooth adapter probe
Basic I/O	PC speaker, I/O ports, key monitor, pager and line editor
Safety net	a clear "kernel panic" screen on a crash instead of a silent freeze

What you can use it for (ideas for you)

These fit someone who builds on an Android phone and wants real low-level skills. Pick one and go.

1. **Study project.** Work through the nanobyte tutorial pages and rebuild each idea inside Chakra, one step at a time (the comparison shows what is left). Write down what you learn in `docs/src/` and rebuild the docs site: your own tutorial grows with the OS.
2. **Portfolio case study.** Take screenshots of the desktop, the Disks and Browser windows, write the problem (build an OS from scratch), the approach (two-stage FAT12 boot, memory manager, scheduler, file system, network stack, desktop), the result (boots in an emulator, runs background jobs, saves files, loads web pages), and link the docs site.
3. **Command and app lab.** Add one command or desktop app a week: a number-base converter, a to-do list stored in a file, a quiz, a clock app, a calculator window, a new background job. It is the fastest way to get good at C.
4. **Phone-only OS dev loop.** In Termux: edit code, `make TOOLCHAIN=clang` , run with `make run-serial` (or VNC for the desktop). No PC needed (see the Android guide).
5. **Network lab.** Write a tiny web server, an echo service or a chat client on top of the TCP code in `kernel/net.c` .
6. **Hardware lab.** Add the xHCI driver, a second network card, or USB keyboard support, using the PCI and UHCI code as templates.
7. **Next milestone: HTTPS.** Add TLS so the browser can open modern sites.
8. **Next milestone: an ELF loader.** Make Chakra run separate programs from the FAT12 disk, using the nanobyte loader as your guide.
9. **Bigger goal: user mode and system calls.** Give programs their own privilege level and address space, so one bad program cannot crash the kernel.
10. **Security learning.** Packet handling, file parsers and drivers are good places to study how bugs become vulnerabilities, and your own OS is a safe place to learn why protections such as paging and user mode exist.
11. **Teach others.** Run a workshop at your school or community: boot Chakra, break it on purpose, fix it.

What it cannot do (yet)

- It is not a daily-use computer: it has a handful of built-in apps and a basic text browser, not a modern web browser or office software.
- It cannot run Linux or Windows programs, or even separate Chakra programs yet (everything is built into the kernel).
- Bluetooth is only a probe of a USB adapter: no scanning, pairing or data yet.
- After a USB boot it cannot save files (the disk driver only sees IDE-style disks).
- It cannot boot directly on an ARM phone, because it is x86. On a phone it runs in an emulator.
- It does not support UEFI-only PCs, HTTPS, Wi-Fi, USB keyboards/mice/sticks, sound, subdirectories or multiple users.
- Its tasks run inside the kernel (no user mode yet), so a buggy task can crash the system.
- It has not been hardened for security, so do not store anything private in it.

Each item is a possible future project, and the roadmap shows a sensible order.

Build Chakra

You do not have to build anything to try Chakra: `dist/chakra.img` is already built. Build it yourself when you want to change the code.

What you need

Tool	Why
`nasm`	assembles the boot sector and entry stubs
`gcc` with 32-bit support, **or** `clang` + `lld`	compiles the C kernel
`ld` and `objcopy` (binutils) with GCC, or `llvm-objcopy` with Clang	links and flattens the kernel
`make`	runs the build
`python3`	builds the FAT12 disk image (`tools/mkimage.py` , no extra packages needed)
`qemu-system-i386`	optional, to run it

Linux (Debian / Ubuntu)

```
sudo apt update
sudo apt install build-essential gcc-multilib nasm make python3 qemu-system-x86
make
```

If `gcc -m32` complains about missing headers you do not need them (Chakra is freestanding), but `gcc-multilib` makes 32-bit linking work reliably. If you prefer Clang:

```
sudo apt install clang lld llvm nasm make python3 qemu-system-x86
make TOOLCHAIN=clang
```

Windows

Use WSL (Windows Subsystem for Linux) with Ubuntu, then follow the Linux steps. To run the image you can use QEMU for Windows from the official site, pointing it at the `build/chakra.img` file inside WSL (`\\wsl$\Ubuntu\home\...`).

macOS

```
brew install nasm qemu make llvm lld python
make TOOLCHAIN=clang
```

(Homebrew's LLVM is not on your PATH by default; `brew info llvm` shows the line to add.) The default GCC on macOS is really Apple Clang for 64-bit, so use the `TOOLCHAIN=clang` path.

Android (Termux)

Termux's compiler targets ARM, so use the Clang cross-target path:

```
pkg install clang lld llvm nasm make python
make TOOLCHAIN=clang
```

This path is verified on desktop Linux. Termux package details can differ; if it fails, use the prebuilt `dist/chakra.img`.

Build commands

Command	Result
`make`	Builds `build/chakra.img`
`make run`	Builds and starts QEMU with a window and a network card (files you save are written into `build/chakra.img`)
`make run-fresh`	Same, but on a throw-away copy, so the built image stays untouched
`make run-floppy`	Same, but the image is attached as a floppy (tests the CHS loader)
`make run-serial`	Builds and runs in the terminal, no window

Command	Result
`make clean`	Deletes `build/`

What the build does

1. Assembles stage 1 and stage 2 (`boot/*.asm`) to flat binaries.
2. Assembles `kernel/entry.asm` and compiles each `kernel/*.c` file with freestanding flags (no standard library, no stack protector, no SSE).
3. Links everything at address `0x10000` using `kernel/linker.ld` and flattens it to `kernel.bin` .
4. Runs `tools/mkimage.py` , which builds a 1.44 MB **FAT12 disk image**: stage 1 in sector 0, stage 2 in sectors 1 to 8, then `KERNEL.BIN` plus every file in `files/` on the volume.

The network card option lives in the Makefile variable `NET` (default `-nic user,model=rtl8139`); override it with `make run NET=""` to run without networking, or add `-usb -device usb-mouse` to try USB.

To put your own file on the disk image, drop it into `files/` (8.3 name, for example `NOTES.TXT`) and run `make` .

Run Chakra on a PC emulator

An emulator is the safest way to run Chakra: nothing on your computer can be damaged, and you can restart in a second. This guide uses QEMU, with notes for VirtualBox and VMware.

Option 1: QEMU (recommended)

Install QEMU

System	Command
Ubuntu / Debian	`sudo apt install qemu-system-x86`
Fedora	`sudo dnf install qemu-system-x86`
macOS	`brew install qemu`
Windows	Download the installer from qemu.org, then add its folder to your PATH, or run the commands below from inside that folder

Start Chakra with a window

Chakra saves your files into the image, so work on a copy and keep the original safe. From the Chakra folder:

```
cp dist/chakra.img work.img
qemu-system-i386 -drive format=raw,file=work.img -nic user,model=rtl8139
```

A window opens and shows `Press T for text mode...` for half a second, then the **graphical desktop**: a Terminal window with the `chakra>` prompt and a taskbar. Click inside the window and use the keyboard and mouse. Press **T** during that half second to start the classic text mode instead. To release your mouse/keyboard from the window press `Ctrl+Alt+G`.

Start Chakra in your terminal (no window)

```
qemu-system-i386 -drive format=raw,file=work.img -nographic
```

Chakra prints to the serial port as well as the screen, so the shell appears in your terminal and you type there. Arrow keys, history and tab completion work. To quit QEMU press `Ctrl+A` then `X`. Colours do not show in this mode, the colour bar appears as `#` characters, and the status bar and the `ui` desktop are screen-only.

Useful options

Option	Meaning
`-m 64`	Give the machine 64 MB of RAM (Chakra needs at least 4 MB; try different sizes, then run `mem`)
`-fda work.img`	Boot the image as a floppy disk. Works, but there is no ATA disk, so files live in RAM
`-serial stdio`	Keep the window and also connect the shell to your terminal
`-nic user,model=rtl8139`	Give Chakra a network card (the only model it supports). Without it, network commands report "no network card"
`-usb -device usb-mouse -device usb-kbd`	Add USB devices so the `usb` command has something to list
`-vnc 127.0.0.1:1`	Show the desktop over VNC instead of a window
`-d int,cpu_reset -D qemu.log`	Log CPU resets and interrupts when debugging a crash

Shortcut with make

`make run`, `make run-fresh` (throw-away copy), `make run-floppy` and `make run-serial` cover the common cases and rebuild first if the code changed.

Option 2: VirtualBox

VirtualBox wants a disk format it knows, so convert a copy of the raw image:

```
VBoxManage convertfromraw dist/chakra.img chakra.vdi --format VDI
```

Then in VirtualBox: New, name `Chakra`, Type `Other`, Version `Other/Unknown`, 64 MB RAM, **Do not add a virtual hard disk**. Open Settings, System, and turn **Enable EFI off** (Chakra needs BIOS). Open Storage and add the disk

`chakra.vdi` to the **IDE controller** (not SATA). Chakra's disk driver only sees IDE disks; on a SATA controller it still boots, but files are kept in RAM only. Start the machine. VirtualBox has no RTL8139 network card option, so networking will not work there; the desktop, files and disk tools do.

Option 3: VMware

Convert the image with `qemu-img convert -f raw -O vmdk dist/chakra.img chakra.vmdk`. Create a new virtual machine with "I will install the operating system later", guest OS "Other", firmware **BIOS**, and use the existing `chakra.vmdk` as its disk, attached as an **IDE** disk (not SCSI or SATA) so Chakra can save files. VMware has no RTL8139 card either, so use QEMU when you want the network features.

What you should see

```
Chakra stage 2
Loading KERNEL.BIN...
Starting kernel
```

then the desktop. In the Terminal window you see the pink CHAKRA banner, a bar of seven colours, three green `[ok]` lines (memory, disk, multitasking) and `chakra>`. Try:

```
help
calc (2+3)*7
write notes.txt hello from chakra
cat notes.txt
dhcp
web http://10.0.2.2:8000/
```

Troubleshooting

Problem	What to do
`Chakra: cannot read stage 2` or `Disk read error`	The image is truncated or was copied in text mode. Rebuild with `make clean && make`, or re-copy `chakra.img` in binary mode.
`KERNEL.BIN not found`	The kernel file is missing from the image. Rebuild with `make`.
Network commands say "no network card"	Start QEMU with `-nic user,model=rtl8139`.
Black screen, nothing happens	In VirtualBox/VMware make sure EFI is **off**. In QEMU make sure you used `-drive format=raw,file=...`.
Typing does nothing in the window	Click the window first. In `-nographic` mode type in the terminal, not a window.
Shell says "RAM disk" instead of "FAT12"	The disk is not on an IDE controller (or you used `-fda`). Attach it as an IDE hard disk.

Problem	What to do
My saved files are gone	You started from a fresh copy of the image. Files are saved inside the image file you booted.
Red `KERNEL PANIC` screen	A CPU exception. Note the exception name and `EIP`, then see the debugging notes in Extending Chakra.

Run Chakra on a real PC from a USB stick

This puts Chakra on a USB stick and boots a physical computer from it. Read the warnings first.

> **Warning 1: writing the image erases the stick.** Everything on the USB stick is overwritten. Use an empty stick. Choosing the wrong device letter can destroy your computer's own disk, so double check it.
>
> **Warning 2: this build has been tested in emulators, not yet on real hardware.** Chakra does not write to your computer's disks, so the risk is low, but a particular PC may not boot it. Try it on a spare machine first.

What the PC needs

- An x86 PC that can boot in **legacy BIOS mode** (called "Legacy", "CSM" or "BIOS" in its settings). Almost any PC from before about 2020 qualifies, and many newer ones offer a compatibility mode. A pure-UEFI machine with no legacy option cannot boot Chakra.
- A USB stick of any size (the image is only 1.44 MB).
- A keyboard. A built-in keyboard works. For an external USB keyboard, enable "USB legacy support" in the BIOS if the keys do not respond.

Step 1: Write the image to the stick

On Linux

1. Plug in the stick and find its device name:

   ```
   lsblk -o NAME,SIZE,MODEL,TRAN
   ```

   Look for the line with `usb` in the last column, for example `sdb`. Use the whole device (`/dev/sdb`), not a partition (`/dev/sdb1`).
2. Unmount it:

   ```
   sudo umount /dev/sdb* 2>/dev/null
   ```

3. Write the image (replace `sdb`):

   ```
   sudo dd if=dist/chakra.img of=/dev/sdb bs=1M conv=fsync status=progress
   sync
   ```

On macOS

```
diskutil list
diskutil unmountDisk /dev/diskN
sudo dd if=dist/chakra.img of=/dev/rdiskN bs=1m
diskutil eject /dev/diskN
```

Replace `N` with the number of the USB stick shown by `diskutil list`.

On Windows

Use a tool that can write a raw disk image:

- **Rufus**: choose the stick, click SELECT and pick `chakra.img`, and if asked choose **DD Image mode**, then START.
- **balenaEtcher**: Flash from file, choose `chakra.img`, choose the stick, Flash.

On Android

Writing raw images to USB from a phone is unreliable. Use a PC for this step, or run Chakra in an emulator on the phone instead (see the Android guide).

Step 2: Boot from the stick

1. Plug the stick into the target PC and turn it on.
2. Open the one-time boot menu. The key depends on the maker. Common ones: `F12` (Dell, Lenovo, Acer), `F9` or `Esc` (HP), `F8` or `F11` (Asus, MSI, Gigabyte). Watch for a hint on the first screen.
3. Pick the USB entry. If you see two (for example one starting with `UEFI:`), choose the one **without** `UEFI`. If there is only a UEFI entry, enter the BIOS setup and enable Legacy/CSM boot first, then retry.
4. You should see `Chakra stage 2`, `Loading KERNEL.BIN...`, `Press T for text mode...`, then the graphical desktop with a Terminal window and `chakra>`. Type `help`. If the screen stays black or garbled, restart and press **T** when prompted to use text mode.

> **Files on a USB boot are not saved.** Chakra's disk driver talks to IDE-style disks, and a USB stick is not one, so after a USB boot `disk` says "RAM disk" and your files vanish at reboot. Everything else works: the desktop, shell, tasks and memory manager. The mouse works if the PC has a PS/2 mouse or a touchpad (a USB mouse works when the BIOS emulates PS/2 for it). The network works only on a PC with a Realtek RTL8139 card, which is rare today; `usb` lists devices only on old-style UHCI controllers, so on most modern PCs it reports the controller as detected but not supported. To keep files, run Chakra in an emulator (the image file keeps them).

Step 3: Get your stick back

Chakra only uses the first 1.44 MB of the stick (it is a small FAT12 volume, so Windows or Linux may show the stick as a tiny disk containing `KERNEL.BIN`, `README.TXT` and `WELCOME.TXT`; that is normal). To use the stick normally again, reformat it: on Windows right-click it in Explorer and choose Format; on Linux run `sudo wipefs -a /dev/sdb` and create a new partition table with your disk tool; on macOS use Disk Utility, Erase.

Troubleshooting

Problem	What to do
The stick is not in the boot menu	Legacy/CSM may be off. Enable it in BIOS setup. Also try a different USB port (a USB 2.0 port is more reliable).
`Chakra: cannot read stage 2`	The BIOS is treating the stick in an unusual way. Try the other boot entry for the same stick (for example "USB-HDD" versus "USB-FDD"), or try another stick.
`Disk read error` or `KERNEL.BIN not found`	The image was not written completely. Write it again and run `sync` before unplugging.
Machine restarts or hangs right after the stage 2 messages	Try another PC. Some BIOSes handle the A20 line or memory calls differently; send the exact PC model when asking for a fix.
Keyboard does not respond	Enable "USB legacy support" / "USB keyboard support" in BIOS, or use a PS/2 or built-in keyboard.
Desktop is garbled or black	Restart and press **T** at the prompt to use text mode.
Screen blank but PC is running	Some PCs start the display in a non-text mode. Try a different PC or a different video output.
`disk` says RAM disk	Expected on USB boot, see the note in Step 2.

Run Chakra on an Android phone

An Android phone has an ARM processor, but Chakra is written for x86. So on a phone, Chakra runs **inside an emulator**. This guide gives three ways, best first.

Method 1: Termux + QEMU (recommended)

Step 1: Install Termux

Install **Termux from F-Droid** (f-droid.org). The Google Play version is outdated and cannot install packages.

Step 2: Install QEMU

Open Termux and run:

```
pkg update
pkg upgrade
pkg install qemu-system-i386
```

Answer `y` to prompts.

Step 3: Get Chakra onto your phone

Download or copy `chakra.img` (or the whole `chakra-os.zip`) so it is in your phone's Downloads folder. In Termux:

```
termux-setup-storage
cp ~/storage/downloads/chakra.img ~/chakra.img
cp ~/chakra.img ~/chakra-original.img
```

The second copy is your clean backup: Chakra saves your files **into** `chakra.img`, so they are still there next time you start it. To start over, copy the backup back over `chakra.img`.

Allow storage access when Android asks. If you only have the zip:

```
pkg install unzip
unzip ~/storage/downloads/chakra-os.zip -d ~/
cp ~/chakra-os/dist/chakra.img ~/chakra.img
cp ~/chakra.img ~/chakra-original.img
```

Step 4: Run it (text mode)

```
qemu-system-i386 -drive format=raw,file=chakra.img -nographic -nic user,model=rtl8139
```

Within a few seconds the banner and `chakra>` prompt appear in Termux. Type with your phone keyboard. Press Enter to run a command. The arrow keys on Termux's extra-keys row work for history and editing, and Tab completes names.

- Colours do not show in this mode and the colour bar appears as `#` characters. The graphical desktop, the status bar and the `ui` text desktop are screen-only, so use Step 5 to see them. The network commands (`dhcp`, `ping`, `get`, `web`) work in this mode.
- To exit QEMU press `Ctrl+A`, then `X`. Termux has a `Ctrl` key on its extra-keys row; tap `Ctrl`, then `A`, release, then `X`.
- Try `-m 32` to give it more memory (Chakra needs at least 4 MB). There is no hardware acceleration on Android, so the emulator is software only, which Chakra handles fine.

Step 5 (optional): Run it with a graphical screen

QEMU can serve the screen over VNC, and a VNC app on the same phone shows it:

```
qemu-system-i386 -drive format=raw,file=chakra.img -nic user,model=rtl8139 -vnc 127.0.0.1:1
```

Install a VNC viewer app (for example bVNC or RealVNC Viewer), connect to `127.0.0.1` port `5901`, and you will see the real coloured screen, including the **graphical desktop** with windows. Use your VNC app's touch pointer as the mouse. QEMU on a phone is software-only, so the desktop is slower than on a PC: be patient with window drags, or press **T** at the boot prompt (you will need a keyboard) for the lighter text mode. Switch back to Termux to stop QEMU with `Ctrl+C`.

Method 2: Build it on the phone

If you want to change the code on your phone, build with the Clang path:

```
pkg install clang lld llvm nasm make python
cd ~/chakra-os
make TOOLCHAIN=clang
make run-serial
```

This build path works on desktop Linux. Termux packages vary by device and version; if the build fails, tell me the first error line and use the prebuilt `dist/chakra.img` meanwhile.

Method 3: An Android emulator app

Apps such as **Limbo PC Emulator** run QEMU with a touch interface. In the app: create a new machine, architecture `x86`, machine type `pc`, RAM 64 MB, attach `chakra.img` as **Hard Disk A** (an IDE disk, so Chakra can save files), boot from hard disk, start. This was not tested for this guide, so settings names may differ between app versions.

Troubleshooting

Problem	What to do
`qemu-system-i386: command not found`	Run `pkg install qemu-system-i386` again, then close and reopen Termux.
`chakra.img: No such file`	Run `ls ~` and `ls ~/storage/downloads` to find where the file really is, then copy it.
Nothing appears after starting	Wait ten seconds. Then check you used `-nographic` or `-vnc`; plain `qemu-system-i386` needs a window Termux cannot show.
You cannot get out	Press `Ctrl+A` then `X`. Or close the Termux session from the notification.
Files I saved are gone	You copied the backup over `chakra.img`, or booted a different copy. Saved files live inside the image you boot.
Keys type twice or oddly	Turn off your keyboard's auto-correct / predictive text, or use Termux's own extra-keys row.
Cannot run it directly on the phone hardware	Correct: Chakra is x86 and your phone is ARM. A native phone version would need an ARM port of the kernel (see Extending Chakra).

How to use Chakra, step by step

This is the hands-on walkthrough. It assumes Chakra is already running (see the run guides for your device). In graphics mode you see the desktop with a Terminal window; in text mode (press T at boot) you see the purple status bar, the banner and the `chakra>` prompt. The shell is the same in both.

The basics

- Type a command and press **Enter**. Commands are lowercase words followed by arguments.
- **Left/Right/Home/End** move in the line. **Backspace** and **Delete** remove characters. **Up/Down** bring back earlier commands. **Tab** completes names.
- **Ctrl+C** cancels the line. **Ctrl+L** clears the screen. **Ctrl+U** clears the line.
- If you mistype, Chakra says `command not found (try 'help')`. Nothing breaks.
- A line can be up to 70 characters.
- The status bar on the top row shows version, uptime, free RAM, number of tasks, disk type and the clock. It keeps updating by itself, because it is a separate task.

Lesson 1: Look around (2 minutes)

```
help
help files
about
mem
memmap
cpu
disk
uptime
date
time
```

`help` lists the topics. `mem` shows RAM, free pages, the heap and whether paging is on. `memmap` shows the memory map your BIOS reported. `disk` shows whether Chakra found a disk and which file system it uses.

Lesson 2: Calculator

```
calc 2+2
calc (2+3)*7
calc 100/7
calc 100%7
calc -5*3
calc 10/0
```

`calc` does whole-number maths with `+ - * / %`, brackets and negative numbers. `100/7` gives 14 because there are no decimals. Dividing by zero gives a friendly error, not a crash.

Lesson 3: Files that survive a reboot

```
ls
cat README.TXT
write todo.txt buy bread
append todo.txt call mum
cat todo.txt
cp todo.txt backup.txt
ren backup.txt old.txt
rm old.txt
```

```
ls
reboot
```

After the reboot run `ls` and `cat todo.txt`: the file is still there. Chakra saves files to the FAT12 disk (the image file you booted).

- File names are 8.3: up to 8 letters, a dot, up to 3 letters, any case (`todo.txt`, `NOTES.TXT`). Chakra shows them in capitals.
- If `disk` says "RAM disk" (floppy or USB boot), files are lost at reboot instead.
- A file can be as large as the free space (`df`).

Lesson 4: Send output to a file

```
echo first entry > log.txt
mem >> log.txt
ls >> log.txt
cat log.txt
```

`>` replaces the file and `>>` adds to it. It works for any command.

Lesson 5: Work faster at the prompt

1. Type `echo hello`, press Enter, then `echo world`, press Enter.
2. Press **Up** twice: `echo hello` comes back. Press **Down** to go forward.
3. Press **Left** a few times and type some letters: they are inserted in the middle.
4. Type `ch` and press **Tab**: it completes to `chakras`.
5. Type `cat w` and press **Tab**: it completes the file name `welcome.txt`.
6. Type `ca` and press **Tab**: it lists `calc` and `cat`.
7. `history` lists your last commands.

Lesson 6: Background jobs (multitasking)

```
tasks
bg primes 300000
bg count
tasks
```

`bg` starts a job as its own task while you keep working. `tasks` shows each task's state, how many timer ticks it has used and its progress. Wait a few seconds, run `tasks` again, then:

```
cat primes.txt
kill 3
```

`primes.txt` holds the answer the background job computed and saved on its own. `kill` stops a task by its id; ids 0 (the shell) and 1 (the status bar) are protected.

Lesson 7: The desktop

```
ui
```

A text-mode desktop opens with a menu on the left: **Files**, **System**, **Tasks**, **Chakras**, **About**, **Quit**.

- **Up/Down** choose a menu item. **Right** or **Enter** opens it.
- In **Files**, press Right to browse, Up/Down to pick a file, **Enter** to read it, **Left** to go back.
- **System** and **Tasks** refresh by themselves.
- **Q** or **Esc** returns to the shell.

The desktop draws straight into video memory, so you need a screen: an emulator window, a VNC viewer, or a real PC. It does not appear in a plain `-nographic` terminal.

Lesson 8: Colours and timing

```
color lightgreen black
clear
color yellow blue
color lightmagenta black
chakras
sleep 3
uptime
```

`color` takes a text colour and an optional background (names or numbers 0 to 15; run `color` alone to see the names). `sleep` waits that many seconds (up to 60).

Lesson 9: See a kernel panic on purpose

```
crash null
```

Chakra shows a red KERNEL PANIC screen: "Page fault ... Faulting address (CR2) = 0". That is the memory manager catching a NULL-pointer access. Reboot (or restart the emulator) to continue. Other options: `crash div0`, `crash ud`.

Lesson 10: Add your own command (first real OS change)

1. Open `kernel/shell.c`.
2. Find the function `run()` and add this before the last `else`:

   ```
   else if (!strcmp(c, "hello")) {
       kprintf("Hello, %s! Welcome to Chakra.\n", argc > 1 ? argv[1] : "friend");
   }
   ```

3. Add `"hello"` to the `cmds[]` list near the top (so Tab completes it) and a line to `cmd_help()`.
4. Run `make run-fresh` (or `make run-serial` with no window).

5. At the prompt type `hello Sola`.

You just modified an operating system. For more ideas see Extending Chakra.

Lesson 11: The graphical desktop

Boot normally (do not press T). You get the desktop: a Terminal window and a taskbar with **Terminal, Files, Disks, Editor, Browser, Monitor**.

1. Click **Files** in the taskbar. Click a file, then **Open**: it opens in the **Editor**. Type, press **Ctrl+S** to save.
2. In **Files**, try **New** (type a name like `todo.txt` and press Enter), **Copy**, **Rename** and **Delete**.
3. Drag a window by its title bar. Click the red **x** to close it. Click a window to bring it to the front.
4. Click **Monitor** to watch tasks, memory and network status live.

See the desktop for everything each app can do.

Lesson 12: Look after the disk

```
disk
df
fsck
label MYDISK
stat README.TXT
touch empty.txt
```

`disk` shows the layout and how full the disk is. `fsck` checks the file system; if it reports problems, `fsck fix` repairs them. In the desktop, the **Disks** app does the same with buttons: **Check**, **Repair**, **Format** (asks twice) and **Refresh**.

Lesson 13: Read and edit files in the terminal

```
more WELCOME.TXT
edit notes.txt
```

In `edit`, type `a`, then some lines, then a single `.` on its own to stop adding; `p` lists the lines, `w` saves, `q` quits.

Lesson 14: Go online

Run Chakra with a network card (`make run` does this; with plain QEMU add `-nic user,model=rtl8139`). Then:

```
dhcp
ping 10.0.2.2
nslookup github.com
get http://10.0.2.2:8000/hello.txt hello.txt
web http://10.0.2.2:8000/index.html
```

On your computer, `python3 -m http.server 8000` serves the current folder at `10.0.2.2:8000` from inside Chakra. In the desktop, open the **Browser**, type an address, press Enter; to follow a link, type its number and press Enter. Only `http://` addresses work: HTTPS is not built yet. See internet and the web.

Lesson 15: USB and Bluetooth

Run QEMU with `-usb -device usb-mouse -device usb-kbd`, then:

```
usb
bt
```

`usb` lists the devices on the controller. `bt` looks for a Bluetooth adapter. See USB and Bluetooth.

Lesson 16: Play with the hardware

```
beep 440 300
inb 61
keys
```

`beep` plays a tone on the PC speaker, `inb` reads an I/O port and `keys` shows the code of every key you press (Esc to stop).

Ways to run it, and which keys to use

Where	How you type
Emulator window	Click inside the window to use the mouse and keyboard. `Ctrl+Alt+G` frees your mouse.
`-nographic` / Termux	Type in the terminal; arrow keys and Tab work. The desktop, colours and `ui` do not show. Quit with `Ctrl+A` then `X`.
VNC viewer on Android	Type through the viewer app's keyboard and use the touch pointer as the mouse. The whole desktop shows.
Real PC from USB	Use the PC's keyboard. Enable "USB legacy support" in the BIOS if keys do not work. Files are not saved on USB boots.

Handy habits

- **Back up the image before you experiment.** Chakra writes your files into the image you boot, so keep a clean copy (`cp dist/chakra.img work.img`).
- **Rebuild after every change:** `make` (or `make run-fresh`). It only recompiles what changed.
- **Start clean if something is odd:** `make clean && make`.
- **Print to debug:** use `kprintf("value=%d\n", x);` in your code.
- **Crash screen:** a red `KERNEL PANIC` screen shows the exception name and the address (`EIP`). That is a clue, not a disaster: reboot and read the debugging tips in Extending Chakra.

If something does not work

Problem	Try this
Nothing happens when I type	Click the emulator window first, or type in the terminal for `-nographic`.
A command is "not found"	Check spelling with `help`. Commands are lowercase.
`write` says "bad name"	Use 8.3 names: letters and digits, up to 8, optional dot and up to 3 more.
`error: no space left`	The disk is full. Delete files (`rm`) and check `df`.
My files vanished	You booted from USB or a floppy (RAM disk), or from a fresh copy of the image.
`ui` shows nothing	You are in a text-only terminal. Use a window or VNC.
Strange characters appear	You are in `-nographic` mode and the colour bar shows as `#`. Normal.

The graphical desktop

Chakra 1.0 boots into a graphical desktop with windows and a mouse, as long as the BIOS offers a 32-bit video mode (almost every PC and every emulator does). The desktop is drawn by the kernel itself: no X11, no libraries.

What you see

- A **taskbar** along the bottom with six apps and a clock.
- **Windows** with a title bar: drag the title bar to move a window, click the red **x** to close it, click anywhere on a window to bring it to the front.
- The colour stripe across the top and the mouse pointer.
- The **Terminal** window opens first and runs the same shell as the text mode.

Screen size is chosen at boot: 800x600 if available, otherwise 1024x768, otherwise 640x480.

The apps

App	What it does
Terminal	The `chakra>` shell in a window, with history, tab completion and every command.
Files	The file manager: list files, **Open** (in the Editor), **New**, **Copy**, **Rename**, **Delete**, with a free-space bar. Up/Down and Enter work too. `KERNEL.BIN` is protected from deletion.
Disks	The disk manager: disk size, FAT12 layout, clusters used and free, a space bar, and the buttons **Check** (look for problems), **Repair**, **Format** (erases all files, asks twice, keeps `KERNEL.BIN`) and **Refresh**.
Editor	A text editor for files up to 8 KB: type, arrows, Home/End, Backspace/Delete, **Save** or Ctrl+S.
Browser	Type an address, press Enter (or **Go**). Shows the page as text with a numbered link list; type a link number in the address box and press Enter to follow it. Up/Down scroll.

App	What it does
Monitor	Live tasks, uptime, clock, memory, network and screen status.

Keyboard and mouse

- Keys go to the window on top. In the Terminal they go to the shell.
- The mouse is a standard PS/2 mouse (also what emulators provide). USB mice work through the PC's PS/2 emulation; native USB mouse drivers are not built yet.

Text mode

Press **T** when `Press T for text mode...` appears during boot (you have about half a second) to start the classic text mode, with the status bar and the `ui` text desktop. Chakra also falls back to text mode by itself if no video mode is available.

Running the desktop

Where	How
Emulator window	`make run`, or `qemu-system-i386 -drive format=raw,file=work.img -nic user,model=rtl8139`. Click inside the window to capture the mouse; `Ctrl+Alt+G` releases it.
Android (Termux)	Run QEMU with `-vnc 127.0.0.1:1` and open a VNC viewer app on `127.0.0.1:5901`. Software emulation is slower, so expect the desktop to feel sluggish.
Real PC	Boot from USB; the BIOS video mode is used. Not yet tested by the author.
`-nographic`	The desktop runs but is invisible; the shell still works over the serial console.

Internet and the web

Chakra 1.0 has a small network stack written from scratch, and a text-mode web browser.

What is supported

Layer	Support
Network card	**Realtek RTL8139** (the card QEMU emulates with `model=rtl8139`, and many older PCs)
Link and addressing	Ethernet, ARP, IPv4, **DHCP** (falls back to QEMU's 10.0.2.15 if there is no DHCP answer)
Tools	ICMP **ping**, UDP, **DNS** name lookup
Transport	**TCP** client (one stream at a time per connection, up to 4 connections)
Web	**HTTP/1.0** client with redirects and chunked replies, and an HTML-to-text page viewer

Not supported yet: HTTPS (TLS), Wi-Fi, IPv6, other network cards (Intel e1000, Realtek 8169 and so on), servers (Chakra cannot be connected *to*), and images or scripts on web pages. Many big sites now require HTTPS, so for those Chakra will say so (for example GitHub answers "426 Upgrade Required").

Using it

Start QEMU with a network card (the Makefile does this for you):

```
qemu-system-i386 -drive format=raw,file=work.img -nic user,model=rtl8139
```

Command	What it does
`lspci`	List PCI devices (look for the Realtek network controller)
`net`	Show the network card, MAC address, IP, gateway and DNS
`dhcp`	Ask for an address and show the result
`ping <ip or name>`	Four pings
`nslookup <name>`	Look up an IP address, for example `nslookup github.com`
`get <url> [file]`	Fetch a page; show the start of it, or save it to a file
`web <url>`	Fetch and show a page as text (opens the Browser window on the desktop)

Examples:

```
dhcp
ping 10.0.2.2
nslookup github.com
get http://10.0.2.2:8000/page.txt saved.txt
web http://10.0.2.2:8000/index.html
```

The address `10.0.2.2` is how a QEMU guest reaches the machine it runs on, so you can serve test pages from your own computer (`python3 -m http.server 8000`).

In QEMU's default networking, ping works to the gateway (`10.0.2.2`) but pings to the wider internet time out; that is how QEMU's NAT works, not a Chakra fault. DNS and HTTP work normally.

What was tested

In QEMU with the RTL8139: DHCP, ARP, ping to the gateway, DNS lookup of `github.com` (got a real address), HTTP to a local web server (plain text and an HTML page, plus saving to a file), and a real request to `github.com` over the internet (it answered `426 Upgrade Required` , which proves the connection works). Not tested on real network hardware.

Android and Termux

QEMU in Termux provides the same user-mode networking, so the same commands work there when you add `-nic user,model=rtl8139`. A phone's own Wi-Fi is not used directly; QEMU shares the phone's connection with Chakra.

USB and Bluetooth

Chakra 1.0 has the **foundation** for USB and Bluetooth: a USB host controller driver, device discovery, and a first Bluetooth probe. It is a base to build on, not a finished feature.

USB

Part	Status
UHCI controller (USB 1.1)	Supported: reset, schedule, port reset, control transfers
Device enumeration	Reads each device's descriptors: ID, class, name, interfaces and endpoints, and selects its configuration
EHCI, OHCI, xHCI controllers	Detected and reported, **not supported yet**. Most modern PCs have xHCI (USB 3), so on those the `usb` command lists the controller but no devices
Hubs	Recognised, but devices behind a hub are not listed yet
Keyboards, mice, storage over USB	Not driven yet (keyboards and mice work through the PC's PS/2 emulation)

Commands:

```
lspci          # find the USB controller
usb            # list controllers and the devices on the UHCI ports
```

In QEMU, add `-usb -device usb-mouse -device usb-kbd` to see devices. Tested this way: a QEMU mouse and keyboard were listed with their vendor/product IDs and names, and a storage device appeared behind a hub.

Bluetooth

Bluetooth adapters are USB devices, which is why USB comes first. The `bt` command looks for a USB Bluetooth adapter (class E0/01/01), sends it the standard **HCI Reset** command, reads the answer from its event endpoint, then asks for its Bluetooth address.

```
bt
```

Honest status: this code is written to the Bluetooth HCI-over-USB specification, but it has **not been tested against a real adapter**, because the emulator available for testing has no Bluetooth device. It also needs the adapter to sit on a UHCI port. What does *not* exist yet: scanning for devices, pairing, and data links (ACL), plus audio and input profiles. Treat `bt` as a first step.

What comes next

1. An **xHCI** driver (needed for most modern PCs).
2. A **hub** driver.
3. **HID**: native USB keyboards and mice.
4. **Mass storage**: USB sticks as disks, so files can be saved after a USB boot.
5. Bluetooth **inquiry (scan)** and connections, built on the same HCI code.

Shell command reference

The prompt is `chakra>`. Type `help` for a short list, or `help files`, `help system`, `help tasks`, `help keys` for details.

Files

File names are 8.3 (up to 8 letters, a dot, up to 3 letters), any case. With a disk they are saved on it and survive reboot; without a disk they live in a 16-file RAM disk.

Command	What it does	Example
`ls` (`dir`)	List files, sizes and free space	`ls`
`cat <f>` (`type`)	Show a file	`cat README.TXT`
`write <f> <text>`	Create or replace a file	`write todo.txt buy milk`
`append <f> <text>`	Add a line to a file	`append todo.txt call mum`
`cp <a> <b>` (`copy`)	Copy a file	`cp todo.txt backup.txt`
`ren <a> <b>`	Rename a file	`ren backup.txt old.txt`
`rm <f>` (`del`)	Delete a file	`rm old.txt`
`hexdump <f>`	Show the first 128 bytes in hex	`hexdump KERNEL.BIN`
`wc <f>`	Count lines, words and bytes	`wc README.TXT`
`df`	Disk space	`df`

System

Command	What it does	Example
`mem` (`free`)	RAM, free pages, paging state, heap use	`mem`
`memmap`	The BIOS memory map (E820)	`memmap`

Command	What it does	Example
`cpu`	CPU vendor string	`cpu`
`disk`	Boot drive, disk size, file system, space	`disk`
`uptime`	Time since boot	`uptime`
`time`, `date`	Read the hardware clock	`time`
`sleep <seconds>`	Wait (up to 60)	`sleep 3`
`color <fg> [bg]`	Change text colours by name or number 0-15	`color lightgreen black`
`clear` (`cls`)	Clear the screen	`clear`
`chakras`	The seven chakras in colour	`chakras`
`about`, `ver`	Show the banner and version	`about`
`crash div0|null|ud`	Trigger a kernel panic on purpose	`crash null`
`reboot`, `halt`	Restart or stop	`reboot`

Tasks and the desktop

Command	What it does	Example
`tasks` (`ps`)	List tasks, state, CPU ticks, progress	`tasks`
`bg primes <n>`	Count primes up to n in the background; result goes to `PRIMES.TXT`	`bg primes 300000`
`bg count`	A background counter; watch PROGRESS in `tasks`	`bg count`
`kill <id>`	Stop a task (ids 0 and 1 are protected)	`kill 2`
`ui`	Open the text-mode desktop (in graphics mode the desktop is already running)	`ui`

Other

Command	What it does	Example
`echo <text>`	Print text	`echo hello`
`calc <expr>`	Integer maths with `+ - * / %` and brackets	`calc (2+3)*7`
`history`	List recent commands	`history`

Disk and file manager

Command	What it does	Example
`disk`	Disk size, FAT12 layout, clusters, space bar	`disk`
`df`	Used and free space with a bar	`df`
`fsck`	Check the file system for problems	`fsck`
`fsck fix`	Check and repair (frees lost clusters, fixes wrong sizes and broken chains)	`fsck fix`
`label [name]`	Show or set the volume label	`label MYDISK`
`stat <f>`	Size, first cluster, chain length, modified time	`stat README.TXT`
`touch <f>`	Create an empty file	`touch notes.txt`
`format yes`	Erase all files (`KERNEL.BIN` is kept so the disk still boots)	`format yes`

Reading and editing files

Command	What it does	Example
`more <f>` (`less`)	Show a file one page at a time (space = next, q = quit)	`more WELCOME.TXT`
`edit <f>`	Line editor: `p` list, `a` add lines (end with `.`), `i N` insert, `d N` delete, a number replaces a line, `w` save, `q` quit	`edit notes.txt`

Network, web and USB

Command	What it does	Example
`lspci`	List PCI devices	`lspci`
`net` (`ifconfig`)	Network card, MAC, IP, gateway, DNS	`net`
`dhcp`	Get an address	`dhcp`
`ping <ip or name>`	Four pings	`ping 10.0.2.2`
`nslookup <name>`	Look up an IP address	`nslookup github.com`

Command	What it does	Example
`get <url> [file]` (`wget`)	Fetch a page, print the start or save it	`get http://10.0.2.2:8000/a.txt a.txt`
`web <url>` (`browse`)	Fetch a page and show it as text (opens the Browser on the desktop)	`web http://10.0.2.2:8000/`
`usb`	USB controllers and devices	`usb`
`bt` (`bluetooth`)	Probe a USB Bluetooth adapter	`bt`

Basic input and output

Command	What it does	Example
`beep [hz] [ms]`	Sound the PC speaker	`beep 600 200`
`inb <port>`	Read a byte from an I/O port (hex)	`inb 61`
`outb <port> <value>`	Write a byte to an I/O port (hex). Careful: wrong ports can hang the machine	`outb 80 1`
`keys`	Key monitor: shows the code of each key; Esc quits	`keys`

Redirection

Send any command's output to a file with `>` (replace) or `>>` (add):

```
ls > files.txt
echo first > log.txt
mem >> log.txt
```

Editing keys

Key	Action
Left / Right, Home / End	Move in the line (also `Ctrl+A` start, `Ctrl+E` end)
Backspace, Delete	Remove characters
Up / Down	Command history (the last 16 commands)
Tab	Complete a command name or a file name (if several names match, they are listed)
`Ctrl+U`	Clear the line

Key	Action
`Ctrl+C`	Cancel the line
`Ctrl+L`	Clear the screen

A line can hold up to 70 characters.

Colour names

`black blue green cyan red magenta brown lightgrey darkgrey lightblue lightgreen lightcyan lightred lightmagenta yellow white`

Chakra vs nanobyte OS

You asked which is better. Short answer: they are built for different jobs, and the best result is to grow Chakra with nanobyte's ideas. This comparison is based on reading the nanobyte source; I did not build or run nanobyte. The Chakra column was built and tested.

	nanobyte OS	Chakra OS 1.0
Goal	A tutorial: teach each piece of OS development step by step	A usable mini OS that boots to a shell
Size	About 4,900 lines of C and assembly	About 6,000 lines including tools
Boot	Two stages; stage 2 is written in C	Two stages, both in assembly; stage 2 is about 900 bytes
Disk format	Stage 2 reads FAT12, FAT16 and FAT32, with long file names and partitions (MBR)	FAT12 only, 8.3 names, whole-disk volume; stage 2 finds `KERNEL.BIN` through the FAT chain
Kernel format	ELF loader	Flat binary at a fixed address
Boot information	Boot parameters with memory regions	BootInfo with the full E820 map
CPU setup	GDT, IDT, PIC, a hardware abstraction layer	GDT, IDT, PIC, PIT timer, panic screen
Memory	Memory map only	Page allocator, paging with NULL guard, heap
Multitasking	No	Preemptive scheduler, background jobs
Disk and files	Loader reads FAT; kernel file system is an outline	ATA driver, FAT12 read/write in the kernel, survives reboot
What it does when it boots	Prints log lines and stops	Graphical desktop with windows and a mouse, plus a shell with about 60 commands

	nanobyte OS	Chakra OS 1.0
Keyboard and mouse	Not in the kernel yet	Interrupt-driven keyboard with arrow keys, PS/2 mouse
Networking	No	RTL8139 driver, TCP/IP, DHCP, DNS, HTTP, web viewer
USB	No	UHCI driver, device discovery, Bluetooth probe
To build it	A cross-compiled GCC, SCons, mtools and more	`nasm`, `gcc` (or `clang` + `lld`), `make`, `python3`
Build on a phone	Impractical	Documented (Termux with Clang), plus a prebuilt image

What nanobyte still does better

- It is a **step-by-step teaching resource**, with written lessons for every stage.
- Its **ELF kernel loader** and its more general boot parameters are the right model if you want to load programs, not just one kernel.
- Its loader handles **FAT12, FAT16 and FAT32, long file names and MBR partitions**, and its code is written in C and organised like a bigger project, with a hardware abstraction layer.

What Chakra does better

- It is **usable straight away**, in an emulator or from a USB stick.
- It has **more of the operating system**: memory management, multitasking, files that persist, and a real shell.
- It is **simpler to build and to read**, which suits learning and phone-only work.

What to take from nanobyte next

1. An **ELF loader**, so Chakra can run separate programs from the FAT12 disk.
2. **FAT subdirectories, long file names and FAT16/32**, so Chakra can use larger disks.
3. A **hardware abstraction layer** if you ever port Chakra to ARM or UEFI.